

课程论文：基于 AI 辅助开发子网划分计算器的实践报告

生成式人工智能辅助网络工程学习与实践

摘要

本文围绕“生成式 AI 辅助网络工程学习与实践”这一主题，选择 IPv4 子网划分作为具体场景，完成了一个可运行的子网划分计算器网页。实践过程中，我使用生成式 AI 辅助拆解需求、整理网页代码结构、解释 JavaScript 位运算和规划验证用例，但没有直接把 AI 输出作为最终结果提交，而是结合网络工程知识对地址块大小、CIDR 借位、网络地址、广播地址和可用主机范围进行了人工验证。最终成果包括单文件 HTML 工具、运行测试结果和说明报告。通过本次实践，我更清楚地理解了 AI 在学习中的价值：它适合帮助我快速形成思路、发现遗漏和改进表达，但最终判断仍必须依靠专业知识、运行结果和可复现的验证过程。

一、选题背景

子网划分是网络工程学习中的基础内容，也是很多同学容易混淆的知识点。学习时经常会遇到几个问题：第一，CIDR 前缀、子网掩码和地址数量之间的关系比较抽象；第二，借几位才能满足指定子网数量，需要把十进制地址和二进制位数联系起来；第三，网络地址、广播地址和可用主机范围的边界容易算错。传统学习方式通常依赖课本例题和手工计算，虽然能训练基本能力，但在反复验证不同例子时效率不高。

因此，我选择用 AI 辅助开发一个子网划分计算器。一方面，这个任务和网络工程专业知识紧密相关，能够体现 AI 对专业学习的支持；另一方面，子网划分结果具有明确的验证标准，不容易停留在“看起来正确”的层面。只要给出一个 IP 地址、原始 CIDR 和子网数量，就可以用网络地址块大小、广播地址规则和主机数量公式进行人工复核。

二、相关知识介绍

IPv4 地址由 32 位二进制组成，通常写成四段十进制，例如 192.168.10.0。CIDR 表示网络前缀长度，例如 /24 表示前 24 位是网络位，后 8 位是主机位。一个 /24 网络共有 2^8 个地址，也就是 256 个地址。进行子网划分时，需要从主机位中借出若干位作为新的子网位。借 1 位可以得到 2 个子网，借 2 位可以得到 4 个子网，借 3 位可以得到 8 个子网。

每个子网的地址数量由新的前缀长度决定。例如把 192.168.10.0/24 划分为 4 个子网，需要借 2 位，新前缀是 /26。/26 还剩 6 位主机位，因此每个子网有 64 个地址。一般情况下，第一个地址是网络地址，最后一个地址是广播地址，中间地址才是可分配给主机的地址，所以可用主机数为 $64 - 2 = 62$ 。这个规则也是本工具进行专业验证的基础。

三、AI 辅助过程

在开始写代码前，我先向 AI 描述了需求：希望制作一个单文件网页，输入 IP、CIDR 和子网数量，输出每个子网的网络地址、广播地址、可用主机范围和可用主机数，并且界面要适合课程展示。AI 给出的初步建议是使用 HTML 表单收集输入，用 JavaScript 把 IPv4 地址转换为 32 位整数，再通过位运算计算网络地址和地址块边界。

随后我继续追问了两个细节。第一，如果用户输入的子网数量不是 2 的整数次方怎么办？AI 提醒需要使用向上取整的 $\log_2$ 计算借位数，例如需要 3 个子网时，实际应借 2 位得到 4 个子网。第二，如何避免输出错误地址？AI 建议加入输入校验，包括 IPv4 必须有四段、每段在 0 到 255 之间、CIDR 在 0 到 32 之间、子网数量必须为正整数。经过人工判断，这些建议符合网络工程和程序设计的基本要求，因此被纳入最终实现。

AI 也有需要修正的地方。例如它最初给出的说明中只强调“满足用户输入的子网数量”，但没有充分解释“实际可划分数目可能大于输入数量”。如果不补充这一点，用户看到输入 3 却输出 4 个子网时可能会误以为程序错误。因此我在网页中增加了提示信息，明确说明子网划分依赖二进制借位，实际子网数必须是 2 的整数次方。

四、实践成果

最终完成的成果是 `作业二_子网划分计算器.html`。网页左侧是输入区域，包含 IPv4 地址、原始 CIDR 和子网数量；右侧是结果区域，包含原始网络、新 CIDR、实际可划分子网数、每个子网地址数，以及详细子网表格。表格列出序号、网络地址、CIDR、可用主机范围、广播地址和可用主机数。

以 192.168.10.0/24 划分为 4 个子网为例，程序输出的新前缀是 /26。第一个子网是 192.168.10.0/26，可用范围为 192.168.10.1 到 192.168.10.62，广播地址为 192.168.10.63；最后一个子网是 192.168.10.192/26，可用范围为 192.168.10.193 到 192.168.10.254，广播地址为 192.168.10.255。这个结果与手工计算一致。

从实现角度看，工具先把点分十进制 IP 转换为 32 位整数，再用 CIDR 生成原始网络掩码，得到原始网络地址。之后根据需要的子网数量计算借位数和新 CIDR，用新的子网块大小逐段递增，依次计算每个子网的起始地址和结束地址。最后再把整数地址转换回点分十进制显示给用户。

五、结果分析与人工验证

为了验证工具的正确性，我使用了典型案例 192.168.10.0/24 划分为 4 个子网。人工计算过程如下：/24 网络有 8 位主机位，共 256 个地址；4 个子网需要借 2 位，因此新前缀是 /26；每个 /26 子网保留 6 位主机位，共 64 个地址；常规可用主机数为 62。子网块大小为 64，所以网络地址依次为 192.168.10.0、192.168.10.64、192.168.10.128、192.168.10.192。每个广播地址分别是下一段网络地址前一个地址，最后一段广播地址是 192.168.10.255。

工具输出与上述人工计算完全一致，因此可以判断核心计算逻辑可靠。同时，我也检查了输入校验，例如 CIDR 超过 32、IP 地址段超过 255、子网数量小于 1 时，网页会给出错误提示，而不是继续计算。这一点可以减少课堂展示时因为误输入产生的混乱。

不过，本工具仍然有边界和局限。它主要面向基础 IPv4 子网划分学习，没有处理 IPv6，也没有区分点到点链路中 /31 的特殊用法。在普通课程练习中，把 /31、/32 的可用主机数按非负数处理是可以接受的，但在真实网络工程中，仍应根据具体设备和协议规则判断。因此，工具适合辅助学习和快速验证，不应替代网络规划中的完整设计评审。

六、AI 的帮助、局限与改进

本次实践中，AI 最有帮助的地方是加快了需求拆解和代码组织。它能提醒我把输入、计算、输出、校验和说明分开考虑，也能把抽象的位运算转换成比较清楚的步骤。对于不熟悉 JavaScript 位运算的学习者，AI 的解释能够降低入门难度，让我更快地把注意力放回网络工程规则本身。

AI 的局限也很明显。它生成的内容可能在边界条件上不完整，也可能把“理论上成立”的说法写得过于绝对。例如，如果只说每个子网可用主机数一定等于总地址数减 2，就忽略了 /31 等特殊场景。又如，如果没有解释实际子网数量必须是 2 的整数次方，就会让工具输出看起来不符合用户直觉。因此，AI 输出必须经过人工筛选、修改和验证。

如果继续改进这个工具，我会加入子网掩码显示、二进制展开过程、更多测试用例和导出结果功能。还可以增加一个“学习模式”，把每一步借位、块大小、网络地址递增原因展示出来，让工具不仅给答案，也帮助用户理解计算过程。

七、结论

通过本次实践，我完成了一个与网络工程基础知识直接相关的 AI 辅助小工具。相比单纯让 AI 写一段说明，这个任务要求我把 AI 的建议转化为可运行代码，并用网络工程规则验证结果。实践表明，生成式 AI 可以显著提高学习效率，帮助我组织思路、生成初稿和发现遗漏；但它不能替代专业判断。真正可靠的学习成果必须同时具备明确问题、可见成果、运行记录和人工验证。

附录：AI 使用声明

本人在本论文和配套作业中使用生成式 AI 辅助需求分析、代码结构设计、说明文字整理和报告润色。本人已对 AI 生成内容进行阅读、判断、修改和验证，网页工具的关键计算结果已通过手工子网划分方法复核。论文中的观点、实践结果和最终表达由本人负责。